

Class 7: Machine Learning 1

Erica Yue Hu (PID:A17787225)

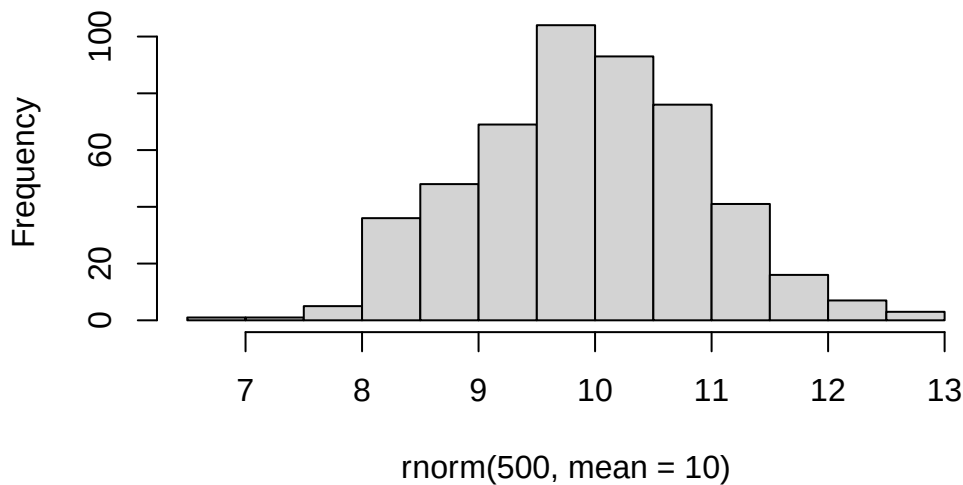
##Background today we will begin our exploration of some important machine learning methods, namely **clustering** and **dimensionality reduction**

Let's make up some input data where we know what the natural groupings are

The function `rnorm()` can be useful here.

```
hist(rnorm(500, mean=10))
```

Histogram of `rnorm(500, mean = 10)`



Q. generate 30 random numbers centered at +3 and another 30 centered at -3

```
tmp<-c(rnorm(30, mean=3), (rnorm(30,mean=-3)))  
cbind(x=tmp, y=rev(tmp))
```

	x	y
[1,]	1.407897	-1.870223
[2,]	1.010090	-2.807037
[3,]	3.446663	-3.784143
[4,]	2.254585	-3.397622
[5,]	4.130182	-3.630000
[6,]	3.379218	-2.210026
[7,]	3.090382	-3.027944
[8,]	4.208698	-1.720487
[9,]	1.425817	-3.810473
[10,]	3.630527	-2.430880
[11,]	3.659788	-3.056392
[12,]	4.252649	-3.141804
[13,]	3.413312	-2.357271
[14,]	2.326304	-4.053024
[15,]	2.256635	-2.995268
[16,]	3.768672	-2.497108
[17,]	3.389749	-3.116841
[18,]	2.587381	-1.878372
[19,]	2.431199	-2.534350
[20,]	2.073355	-3.738703
[21,]	3.833513	-3.142605
[22,]	2.860533	-3.096631
[23,]	2.780457	-2.819845
[24,]	1.579720	-2.978386
[25,]	3.624825	-2.226274
[26,]	3.942476	-3.972635
[27,]	2.291365	-3.042498
[28,]	2.762504	-3.242746
[29,]	2.254651	-4.757832
[30,]	2.930043	-2.242015
[31,]	-2.242015	2.930043
[32,]	-4.757832	2.254651
[33,]	-3.242746	2.762504
[34,]	-3.042498	2.291365
[35,]	-3.972635	3.942476
[36,]	-2.226274	3.624825
[37,]	-2.978386	1.579720
[38,]	-2.819845	2.780457
[39,]	-3.096631	2.860533
[40,]	-3.142605	3.833513
[41,]	-3.738703	2.073355
[42,]	-2.534350	2.431199

```

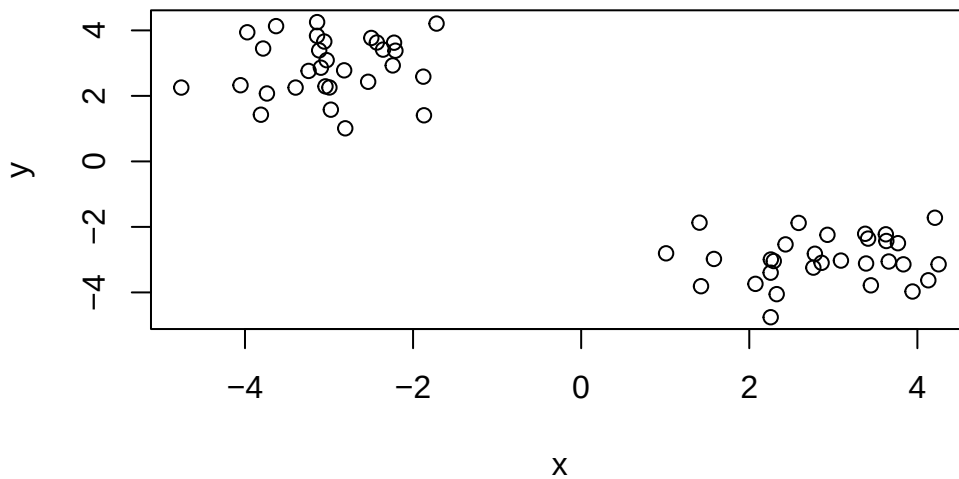
[43,] -1.878372  2.587381
[44,] -3.116841  3.389749
[45,] -2.497108  3.768672
[46,] -2.995268  2.256635
[47,] -4.053024  2.326304
[48,] -2.357271  3.413312
[49,] -3.141804  4.252649
[50,] -3.056392  3.659788
[51,] -2.430880  3.630527
[52,] -3.810473  1.425817
[53,] -1.720487  4.208698
[54,] -3.027944  3.090382
[55,] -2.210026  3.379218
[56,] -3.630000  4.130182
[57,] -3.397622  2.254585
[58,] -3.784143  3.446663
[59,] -2.807037  1.010090
[60,] -1.870223  1.407897

```

```

x<-cbind(x=tmp, y=rev(tmp))
plot(x)

```



rev(): reverse cbind(): binds them as columns rbind(): binds them as rows

##Kmeans clustering

the main function in “base R” for K-means clustering is called `kmeans()`

```
km <- kmeans(x, centers=2)
km
```

K-means clustering with 2 clusters of sizes 30, 30

Cluster means:

	x	y
1	-2.985981	2.900106
2	2.900106	-2.985981

Clustering vector:

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1 1
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Within cluster sum of squares by cluster:

```
[1] 38.05567 38.05567
(between_SS / total_SS = 93.2 %)
```

Available components:

[1]	"cluster"	"centers"	"totss"	"withinss"	"tot.withinss"
[6]	"betweenss"	"size"	"iter"	"ifault"	

Q. what component of the results object details the clusters sizes

```
km$size
```

```
[1] 30 30
```

Q. what component of the results object details the cluster centers?

```
km$centers
```

	x	y
1	-2.985981	2.900106
2	2.900106	-2.985981

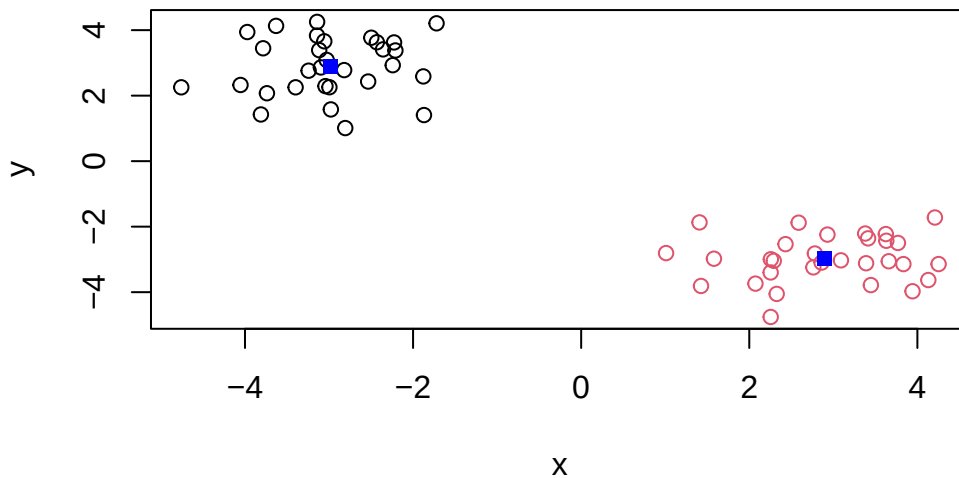
what component of the results object details the cluster membership vector(i.e our main result of which points lie in which cluster)

```
km$cluster
```

```
[1] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 1 1 1 1 1 1 1  
[39] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
```

Q. Plot our clustering results with points colored by cluster and also add the cluster centers as new points

```
plot(x, col=km$cluster)  
points(km$centers, col="blue", pch=15)
```



Q. Run 'kmeans()' again and this time produce 4 clusters *and call your result object k4) and make a results figure like above?

```
k4 <- kmeans(x, center=4)  
k4
```

K-means clustering with 4 clusters of sizes 30, 5, 7, 18

Cluster means:

x y

```
1  2.900106 -2.985981
2 -3.951531  2.066942
3 -2.586591  1.937755
4 -2.873091  3.505788
```

Clustering vector:

[1] 1 4 2 4 3 4 4 3 4
[39] 4 4 2 3 3 4 4 3 2 4 4 4 2 4 4 4 2 4 3 3

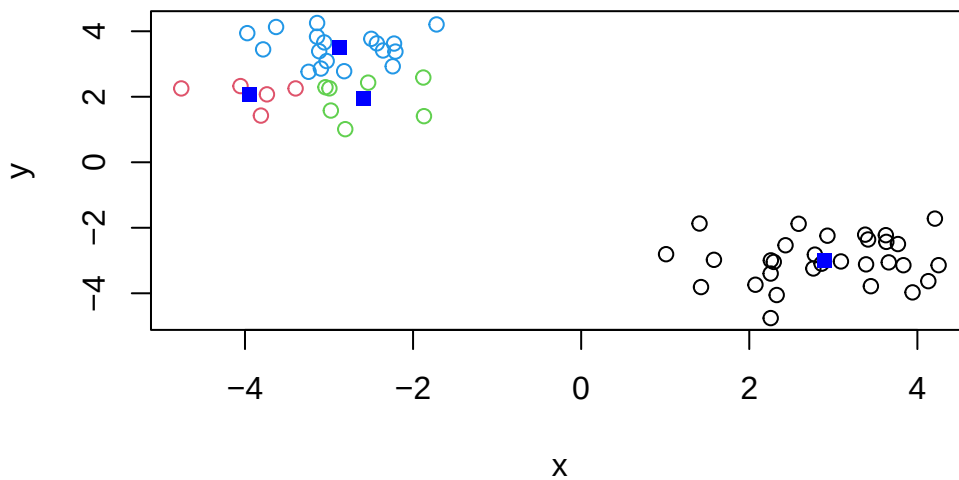
Within cluster sum of squares by cluster:

```
[1] 38.055671  1.581226  3.756180 10.153890
(between_SS / total_SS = 95.2 %)
```

Available components:

```
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

```
plot(x, col=k4$cluster)
points(k4$centers,col="blue", pch=15)
```



the metric

```
km$tot.withinss
```

```
[1] 76.11134
```

```
k4$tot.withinss
```

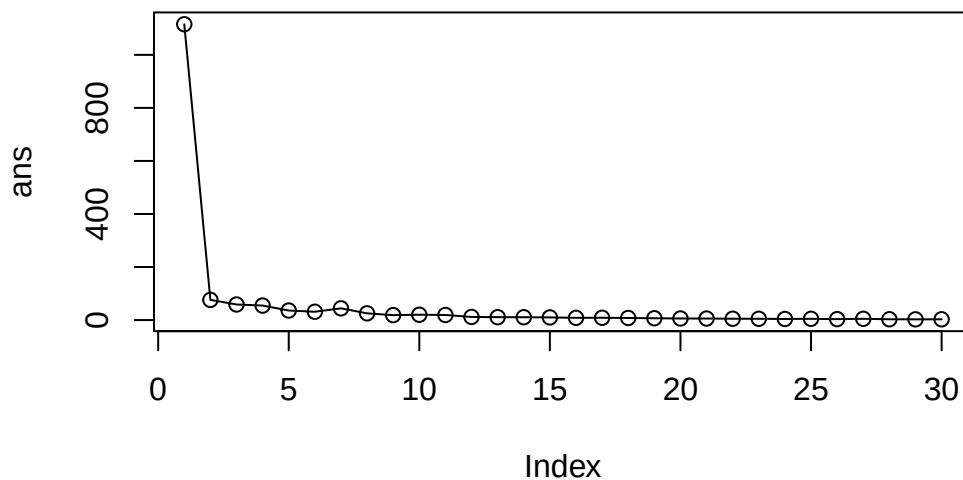
```
[1] 53.54697
```

Q. let's try different number of K(centers) from 1 to 30 and see what the best result is?

```
i <- 1
ans <- NULL
for (i in 1:30) {
  ans <- c(ans, kmeans(x,centers = i)$tot.withinss)
}
ans
```

```
[1] 1115.492131  76.111342  58.723676  54.673837  35.962621  31.328842
[7]  44.230280  25.354498  18.606173  20.101186  19.241226  11.715783
[13]  10.879950  10.652864   9.900632   8.471809   8.686421   7.868018
[19]   6.852717   5.929513   5.724069   4.991902   4.800871   4.167732
[25]   4.726504   3.484509   4.763727   3.014019   2.677173   2.920106
```

```
plot(ans,typ="o")
```



the best result is the point after the drop

key points: k means will impose a clustering structure on your data even if it is not there - it will always give you the answer you asked for even if that answer is silly!

##hierachical clustering

the main function for hierachical clustering is called `hclust()`. Unlike `kmeans()` (which does all the work for you) you can't just pass `hclust()` our raw input data. It needs a "distance matrix" like the one returned from the `dist()` function.

```
d<-dist(x)
hc <- hclust (d)
hc
```

Call:

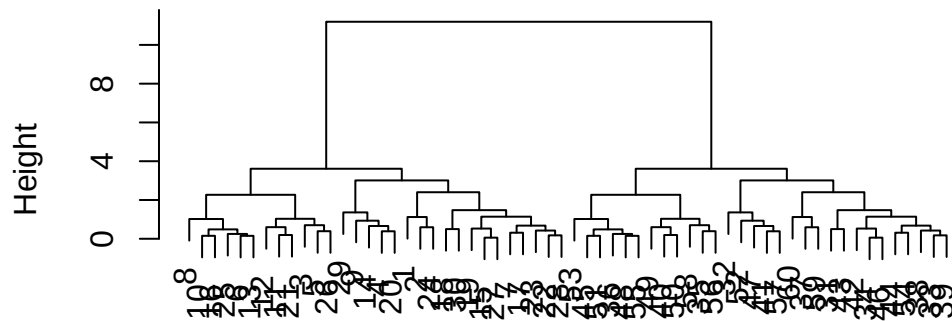
```
hclust(d = d)
```

```
Cluster method   : complete
Distance         : euclidean
Number of objects: 60
```



```
plot(hc)
```

Cluster Dendrogram



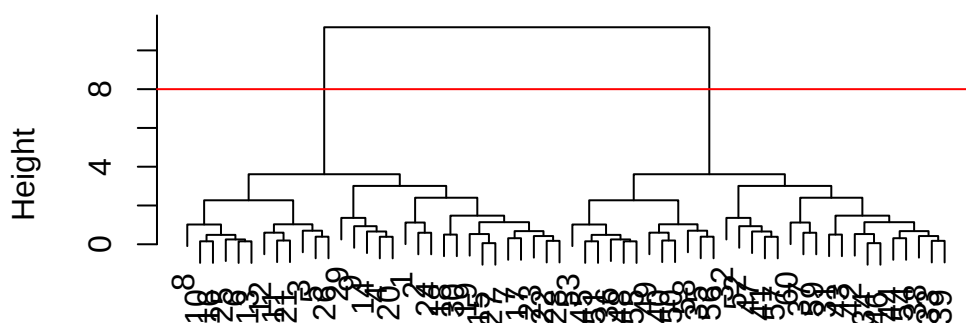
d
hclust (*, "complete")

To extract our cluster membership vector from a `hclust()` result object we have to “cut” our tree at a given height to yield a separate “groups”/“branches”.

```
plot(hc)
abline(h=8, col="red", lty=2)
```

Warning in `int_abline(a = a, b = b, h = h, v = v, untf = untf, ...)`: "lty" is not a graphical parameter

Cluster Dendrogram



d
hclust (*, "complete")

To do this, we use the `cutree()` function on our `hclust()` object:

```
grps <- cutree(hc, h=8)
grps
```

```
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2
[39] 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
```

```
table(grps, km$cluster)
```

```
grps  1  2
      1  0 30
      2 30  0
```

PCA of UK food data

import the dataset of food consumption in the UK

```
url <- "https://tinyurl.com/UK-foods"
x <- read.csv(url)
x
```

	X	England	Wales	Scotland	N.Ireland
1	Cheese	105	103	103	66
2	Carcass_meat	245	227	242	267
3	Other_meat	685	803	750	586
4	Fish	147	160	122	93
5	Fats_and_oils	193	235	184	209
6	Sugars	156	175	147	139
7	Fresh_potatoes	720	874	566	1033
8	Fresh_Veg	253	265	171	143
9	Other_Veg	488	570	418	355
10	Processed_potatoes	198	203	220	187
11	Processed_Veg	360	365	337	334
12	Fresh_fruit	1102	1137	957	674
13	Cereals	1472	1582	1462	1494
14	Beverages	57	73	53	47
15	Soft_drinks	1374	1256	1572	1506
16	Alcoholic_drinks	375	475	458	135
17	Confectionery	54	64	62	41

Q1. how many rows and columns are in your new data frame named x? what r functions could you use to answer this question?

```
dim(x)
```

```
[1] 17  5
```

one solution to set the row names is to do it by hand...

```
#rownames(x)
rownames(x) <- x[,1]
rownames(x)
```

```
[1] "Cheese"          "Carcass_meat "    "Other_meat "
[4] "Fish"            "Fats_and_oils "   "Sugars"
[7] "Fresh_potatoes " "Fresh_Veg "       "Other_Veg "
[10] "Processed_potatoes " "Processed_Veg "   "Fresh_fruit "
[13] "Cereals "        "Beverages"        "Soft_drinks "
[16] "Alcoholic_drinks " "Confectionery "
```

to remove the first column, i can use the minus index trick

```
x <- x[,-1]
x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

A better way to do this is to set the row names to the first column with `read.csv()`

```
x <- read.csv(url, row.names=1)
x
```

	England	Wales	Scotland	N.Ireland
Cheese	105	103	103	66
Carcass_meat	245	227	242	267
Other_meat	685	803	750	586
Fish	147	160	122	93
Fats_and_oils	193	235	184	209
Sugars	156	175	147	139
Fresh_potatoes	720	874	566	1033
Fresh_Veg	253	265	171	143
Other_Veg	488	570	418	355
Processed_potatoes	198	203	220	187
Processed_Veg	360	365	337	334
Fresh_fruit	1102	1137	957	674
Cereals	1472	1582	1462	1494

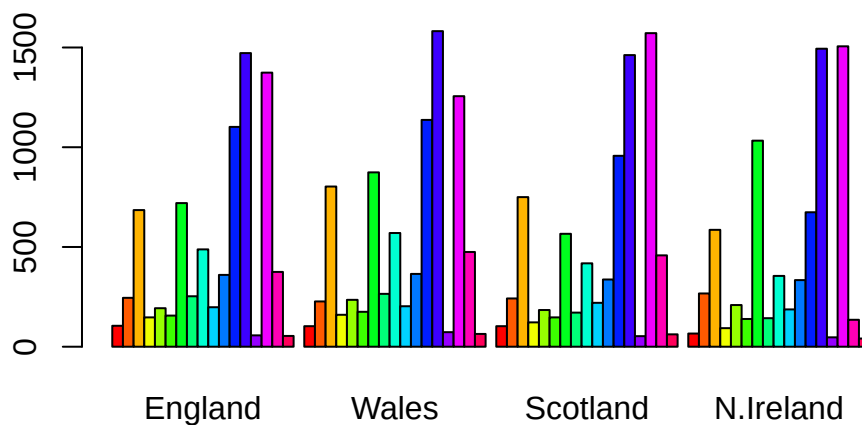
Beverages	57	73	53	47
Soft_drinks	1374	1256	1572	1506
Alcoholic_drinks	375	475	458	135
Confectionery	54	64	62	41

Q2. which approach to solving the “row-names problem” mentioned above do you prefer and why? is one approach more robust than another under certain cricumstances?

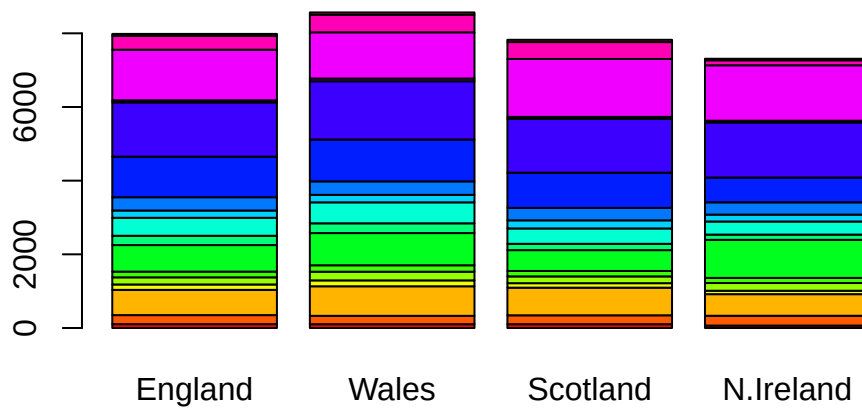
prefer the 2nd method because it doesn't take away a column each time you run the code. also it is shorter.

##spotting major differens and trends is diffuclt even in this wee 17D dataset

```
barplot(as.matrix(x), beside=T, col=rainbow(nrow(x)))
```



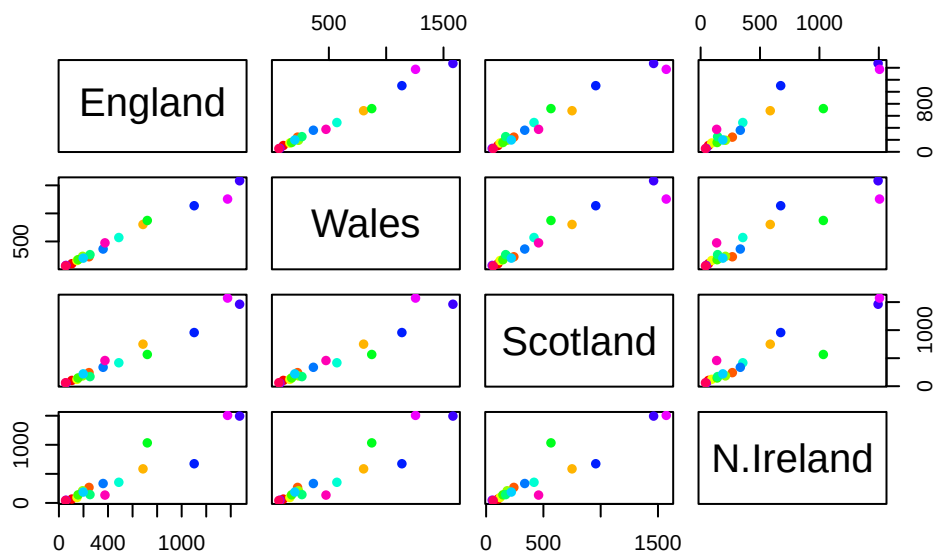
```
barplot(as.matrix(x), beside = F, col=rainbow(nrow(x)))
```



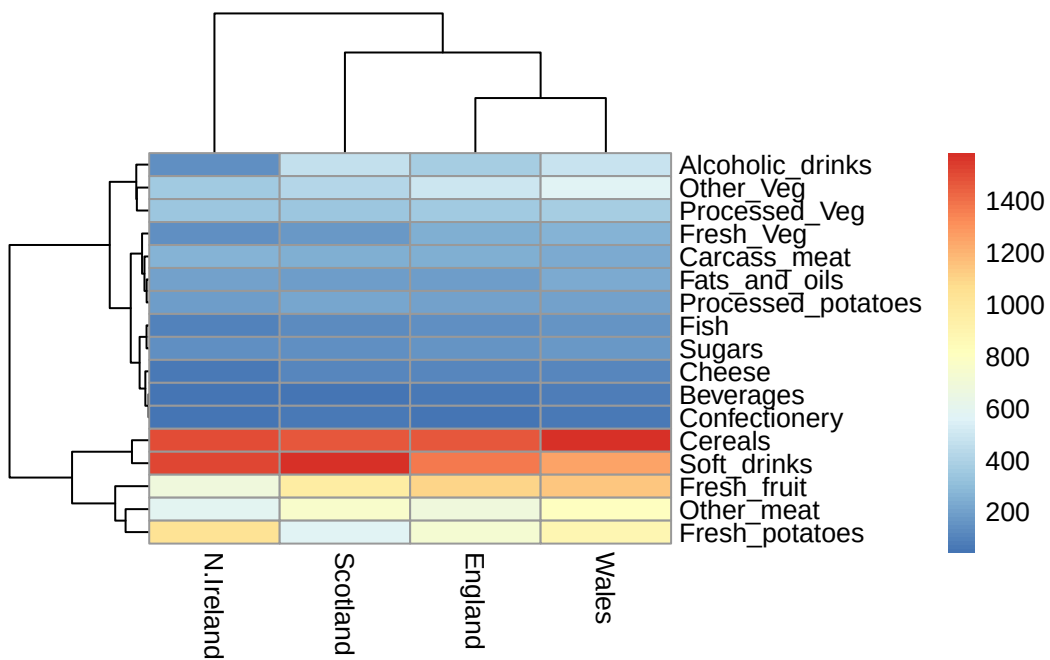
Q. changing what optional argument in the above `barplot()` function results in the following plot?

changing the “beside” argument from T to F changes it from a histogram graph to a stacked graph.

```
pairs(x, col=rainbow(nrow(x)), pch=16)
```



```
library(pheatmap)
pheatmap(as.matrix(x))
```



PCA to the rescue

the main pca function in “base R” is called `prcomp()`. this function wants the transpose of our food data as input (i.e. the foods as columns and the countries as rows)

```
pca <-prcomp(t(x))
```

```
summary(pca)
```

Importance of components:

	PC1	PC2	PC3	PC4
Standard deviation	324.1502	212.7478	73.87622	2.7e-14
Proportion of Variance	0.6744	0.2905	0.03503	0.0e+00
Cumulative Proportion	0.6744	0.9650	1.00000	1.0e+00

```
attributes (pca)
```

\$names

```
[1] "sdev"      "rotation" "center"    "scale"     "x"
```

\$class

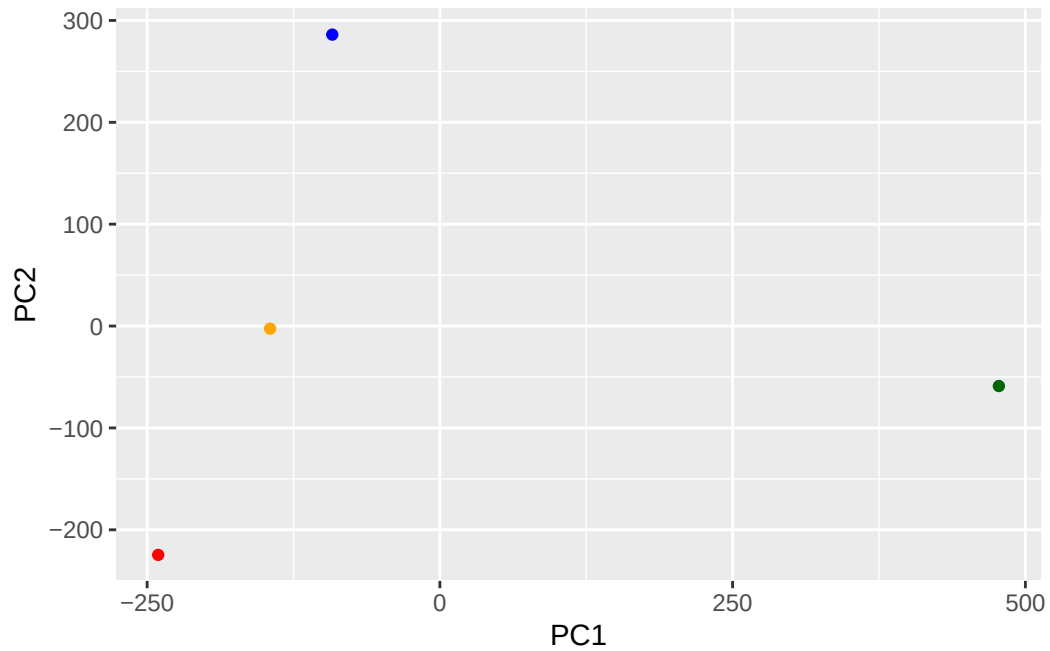
```
[1] "prcomp"
```

To make one of main PCA result figures we turn to ‘pca\$x’ the scores along our new PCs. This is called “PC plot” or “score plot” or “Ordination plot”...

```
my_cols <- c("orange", "red", "blue", "darkgreen")
```

```
library(ggplot2)
```

```
ggplot(pca$x)+  
  aes(PC1,PC2)+  
  geom_point(col=my_cols)
```

the second major result figure is called a “loadings plot” of “variable contributions plot” or “weight plot”.

```
ggplot(pca$rotation)+  
  aes(PC1,rownames(pca$rotation))+  
  geom_col()
```

